DATE: 22/10/2026

CO2-AT1- ANALYTICAL PROBLEM SOLVING

COURSE CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REG NO: 192324235 **NAME:** JOTHIKA M

ANSWERS:

Q1. Morphological Analysis for Document Indexing

Problem Statement

Design a morphological analysis pipeline for the input words connected, connecting, and connection to identify their roots and suffixes, classify the suffixes as inflectional or derivational, normalize the words to a common base form, and display the parsed structure and normalized output using a Python program.

1. MORPHOLOGICAL DECOMPOSITION

Input word	Root	Suffix	Type	Normalized form
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

2. EXPLANATION

- connected: connect + ed — past-tense form of the verb.
- connecting: connect + ing — present participle or gerund form.
- connection: connect + ion — forms a noun from the verb.

1. connected = connect + ed

- connect is the root.
- -ed indicates past tense and is inflectional.
- Normalized form: connect.

2. connecting = connect + ing

- connect is the root.

- -ing indicates a participle or gerund and is inflectional.
- Normalized form: connect.

3. connection = connect + ion

- connect is the root.
- -ion forms a noun and is derivational.
- Normalized form: connect.

The suffixes -ed and -ing are inflectional because they express grammatical forms. The suffix -ion is derivational because it forms a noun.

CODE

```
#jothika-192324235

words = ["connected", "connecting", "connection"]

print(f"{'Word':<15}{'Root':<12}{'Suffix':<10}{'Type':<16}{'Normalized'}")

print("-" * 65)

for word in words:

    if word.endswith("ed"):

        root, suffix, kind = word[:-2], "-ed", "Inflectional"

    elif word.endswith("ing"):

        root, suffix, kind = word[:-3], "-ing", "Inflectional"

    elif word.endswith("ion"):

        root, suffix, kind = word[:-3], "-ion", "Derivational"

    else:

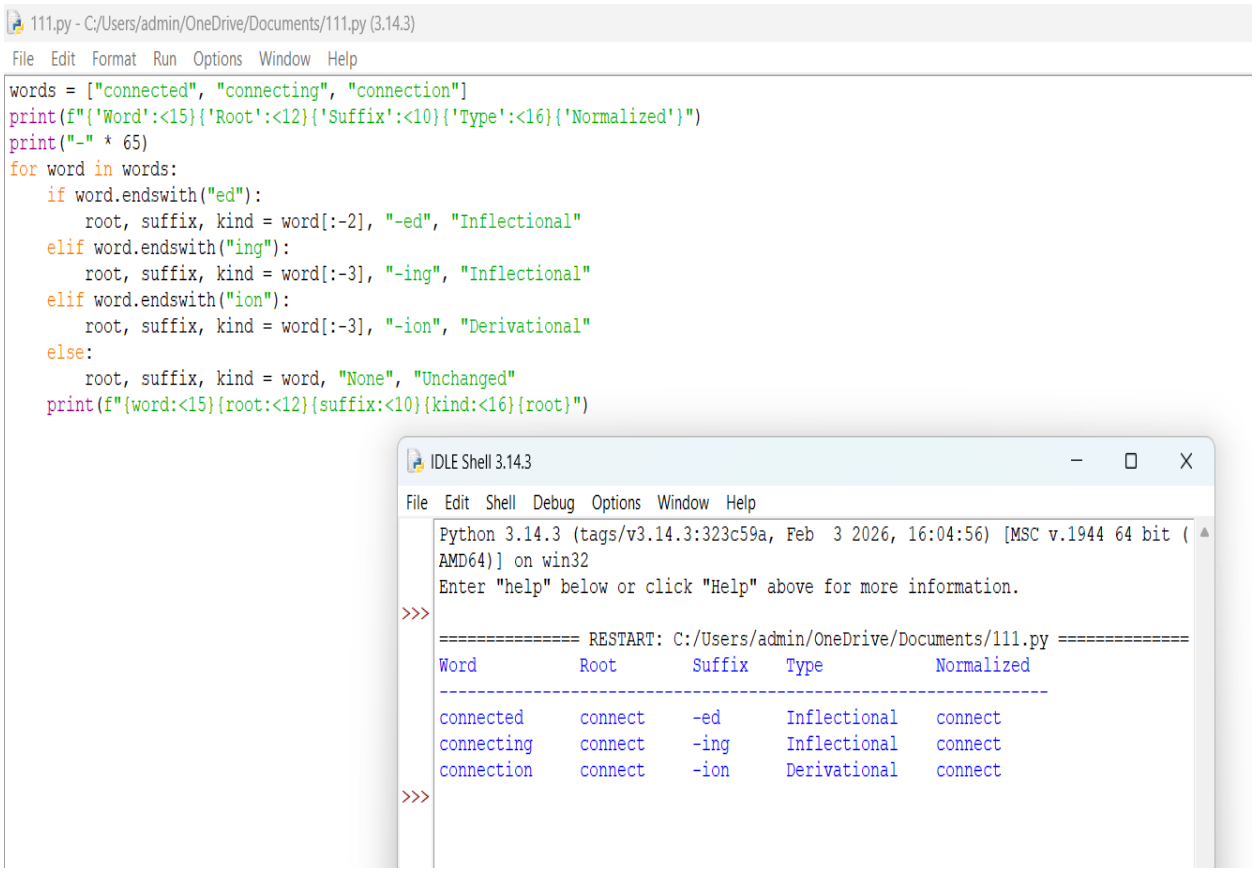
        root, suffix, kind = word, "None", "Unchanged"

    print(f"{word:<15}{root:<12}{suffix:<10}{kind:<16}{root}")
```

EXPECTED OUTPUT

Word	Root	Suffix	Type	Normalized
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

OUTPUT



Conclusion: The three words are normalized to connect, enabling a document indexing system to retrieve documents containing these related forms.

Q2. Morphological parsing of unhappy, happiness, and happily

Problem Statement

Design a rule-based morphological parsing module for sentiment analysis using unhappy, happiness, and happily. Identify prefixes, suffixes, and base forms; classify the transformations as inflectional or derivational; preserve their semantic relationships; and display the morphological breakdown and normalized root using Python.

1. MORPHOLOGICAL ANALYSIS

Word	Prefix	Base	Suffix	Classification	Normalized root
unhappy	un-	happy	None	Derivational	happy
happiness	None	happy	-ness	Derivational	happy
happily	None	happy	-ly	Derivational	happy

2. STEP-BY-STEP DECOMPOSITION

1. unhappy = un- + happy
    - Prefix: un-
    - Base/root: happy
    - un- expresses negation.
    - Classification: derivational.
    - Normalized root: happy.
2. happiness = happy + -ness
    - The spelling changes from y to i before adding -ness.
    - Surface form: happi + ness.
    - Morphological base: happy.
    - -ness converts an adjective into a noun.
    - Classification: derivational.
    - Normalized root: happy.
3. happily = happy + -ly
    - The spelling changes from y to i before adding -ly.
    - Surface form: happi + ly.
    - Morphological base: happy.
    - -ly converts an adjective into an adverb.
    - Classification: derivational.

All three transformations are derivational, not inflectional. Their semantic meanings are related to the concept of happy, although unhappy expresses the opposite sentiment.

CODE

```
#jothika-192324235

words = ["unhappy", "happiness", "happily"]

print(f"{'Word':<15}{'Prefix':<10}{'Base':<10}{'Suffix':<10}{'Type':<15}{'Root'}")

print("-" * 75)

for word in words:

    if word == "unhappy":

        prefix, base, suffix = "un-", "happy", "None"

    elif word == "happiness":

        prefix, base, suffix = "None", "happy", "-ness"

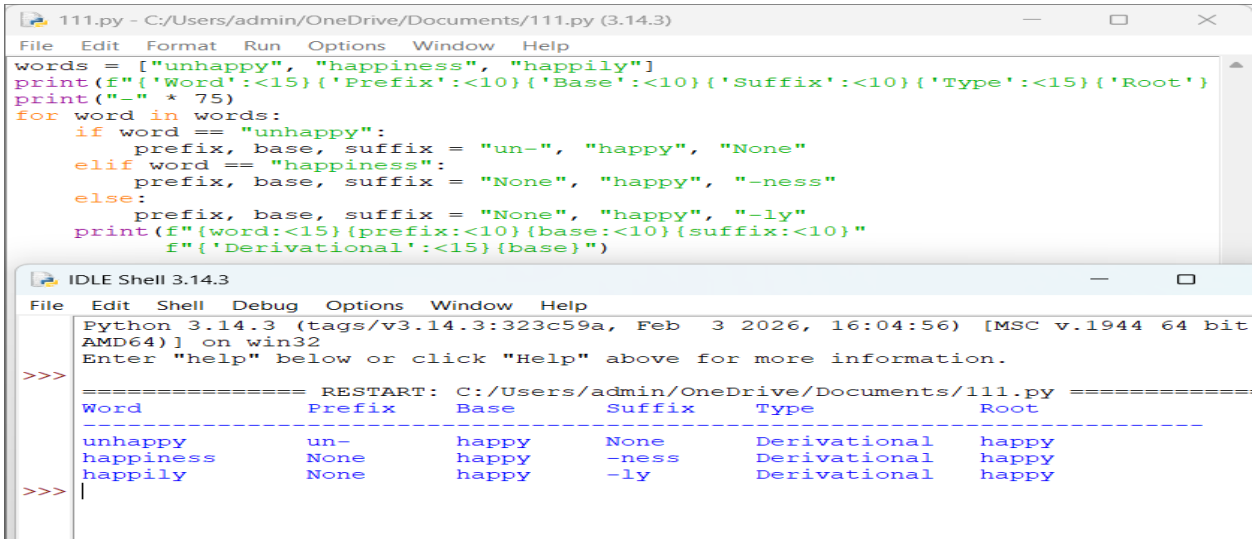
    else:

        prefix, base, suffix = "None", "happy", "-ly"

    print(f"{word:<15}{prefix:<10}{base:<10}{suffix:<10}"

          f"{'Derivational':<15}{base}")
```

OUTPUT



Conclusion: All three words share the normalized root happy. However, unhappy expresses negative sentiment, so a sentiment analysis system should preserve the negation instead of treating all three words as having identical sentiment.

Q3. Stemming-Based Preprocessing of played, player, and playing

Problem statement

Develop a stemming-based preprocessing module for a search engine using played, player, and playing. Apply morphological rules to remove suffixes, identify the stem, distinguish grammatical inflections from word-formation changes, and display the original word, extracted stem, removed affix, transformation type, and normalized form using Python.

1. MORPHOLOGICAL ANALYSIS

Word	Decomposition	Removed affix	Classification	Stem
played	play + ed	-ed	Inflectional	play
player	play + er	-er	Derivational	play
playing	play + ing	-ing	Inflectional	play

2. STEP-BY-STEP DECOMPOSITION

1. played = play + ed
    - Remove -ed.
    - -ed represents the past tense.
    - Classification: inflectional.
    - Normalized form: play.
2. player = play + er
    - Remove -er.
    - -er forms a noun referring to a person who plays.
    - Classification: derivational.
    - Normalized form: play.
3. playing = play + ing
    - Remove -ing.
    - -ing indicates a participle or gerund.
    - Classification: inflectional.
    - Normalized form: play.

CODE:

```
#jothika-192324235

words = ["played", "player", "playing"]

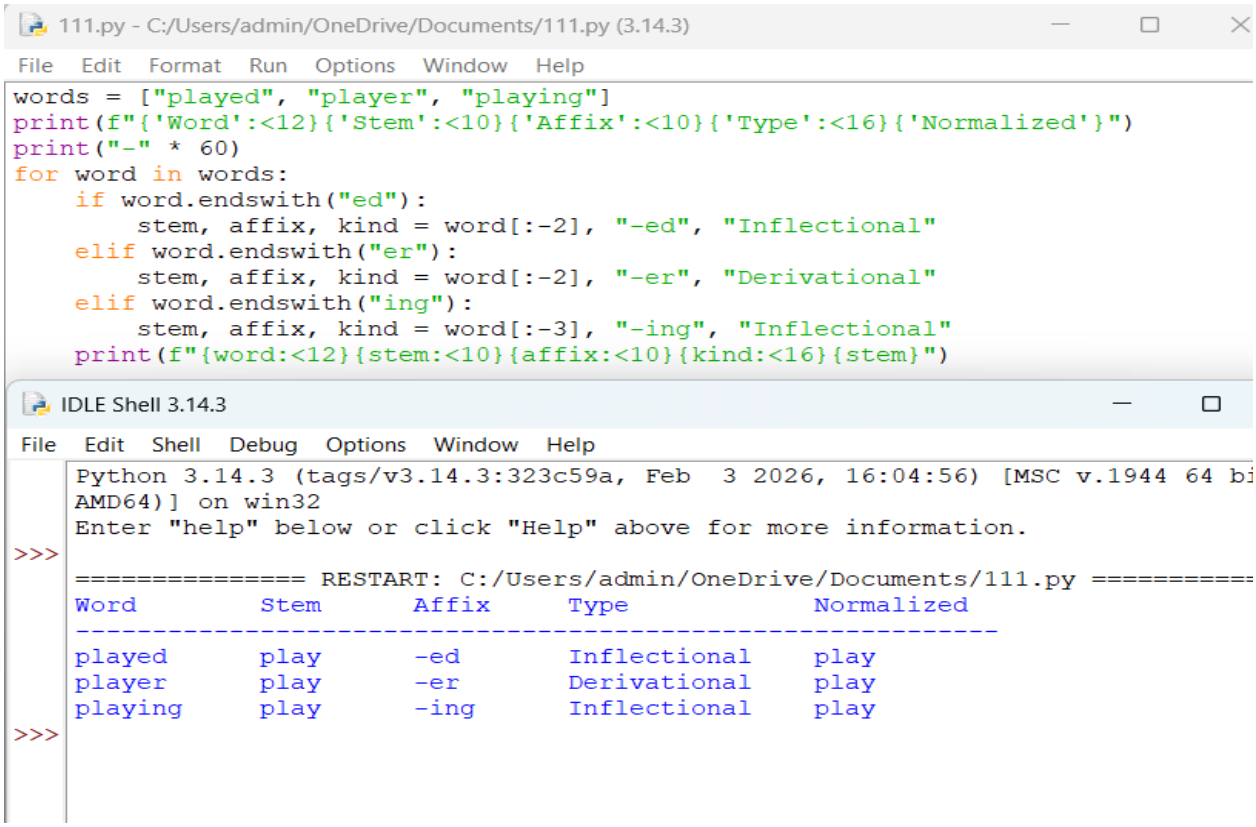
print(f"{'Word':<12}{'Stem':<10}{'Affix':<10}{'Type':<16}{'Normalized'}")

print("-" * 60)

for word in words:
```

```
    if word.endswith("ed"):
        stem, affix, kind = word[:-2], "-ed", "Inflectional"
    elif word.endswith("er"):
        stem, affix, kind = word[:-2], "-er", "Derivational"
    elif word.endswith("ing"):
        stem, affix, kind = word[:-3], "-ing", "Inflectional"
    print(f"{word:<12}{stem:<10}{affix:<10}{kind:<16}{stem}")
```

OUTPUT:



Conclusion: All three words are normalized to play, improving search results when users search for documents containing different forms of the word.

Q4. Finite-State Morphological Parsing of writes, writing, and written

Problem statement

Develop a finite-state morphological parser for writes, writing, and written. Construct state transitions to process regular suffixes and irregular verb forms, identify the root, classify the inflectional patterns, trace the transition paths, and display the morphological components and normalized representations using Python.

1. MORPHOLOGICAL ANALYSIS

Word	Root	Morphological breakdown	Pattern	Normalized form
writes	write	write + s	Regular	write
writing	write	write + ing; final e dropped	Regular	write
written	write	Irregular past participle	Irregular	write

2. FINITE-STATE TRANSITION MODEL

The parser uses the following states:

- q0: Start state.
- q1: Root or regular verb pattern recognized.
- q2: Regular suffix processed.
- q3: Irregular form identified using a lookup table.
- qf: Final state; normalized root produced.

Input	State transition	Processing
writes	q0 → q1 → q2 → qf	Remove s
writing	q0 → q1 → q2 → qf	Remove ing and restore e
written	q0 → q3 → qf	Look up irregular form

3. STEP-BY-STEP DECOMPOSITION

1. writes
    - Remove -s.
    - Root: write.
    - Classification: regular inflection.
2. writing
    - Remove -ing to obtain writ.
    - Restore the final e to obtain write.
    - Classification: regular inflection.
3. written
    - Ordinary suffix removal does not reliably produce write.
    - Use the irregular-form dictionary: written → write.
    - Classification: irregular inflection.

CODE

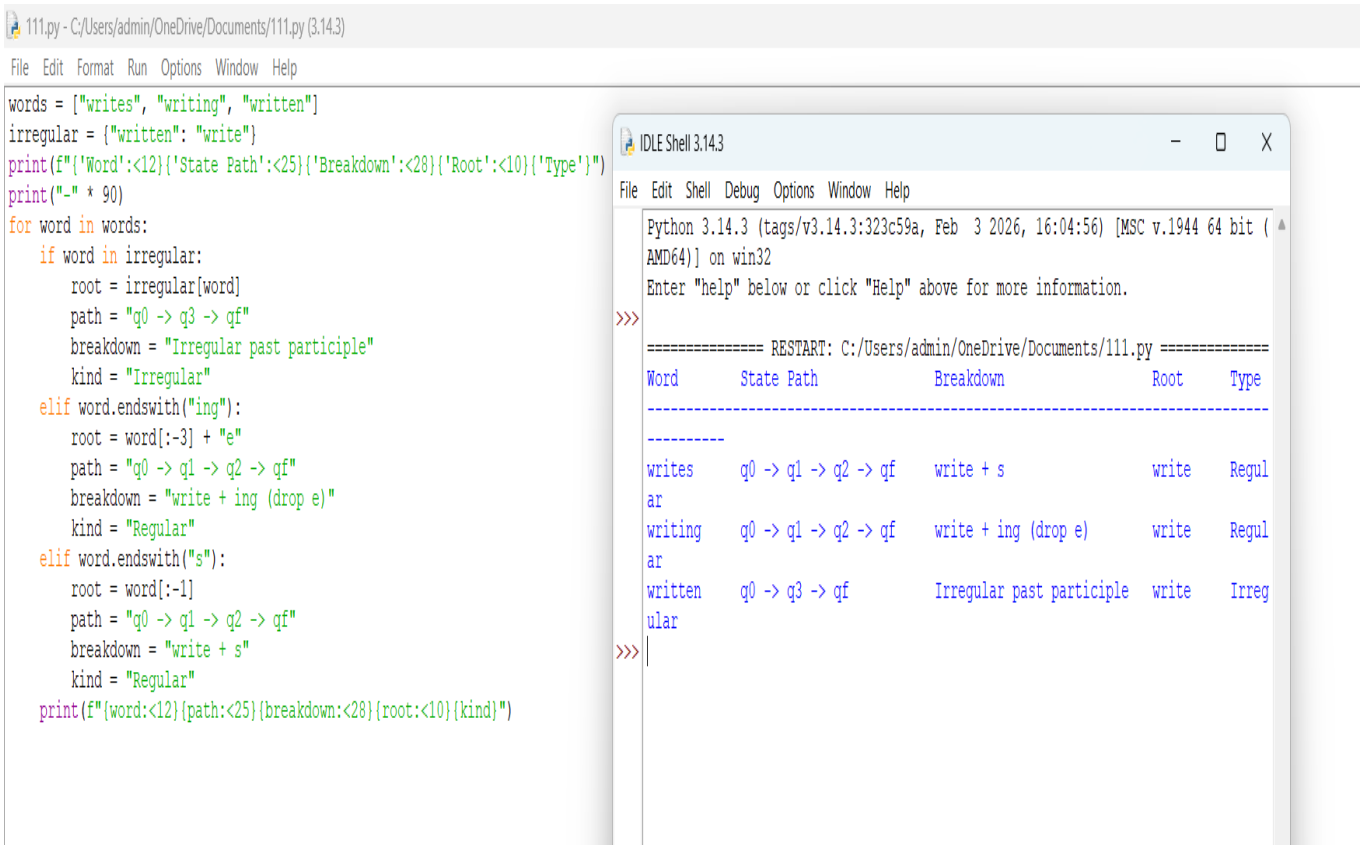
#jothika 192324235

```
words = ["writes", "writing", "written"]
irregular = {"written": "write"}
print(f"{'Word':<12}{'State Path':<25}{'Breakdown':<28}{'Root':<10}{'Type'}")
print("-" * 90)
for word in words:
    if word in irregular:
        root = irregular[word]
        path = "q0 -> q3 -> qf"
        breakdown = "Irregular past participle"
        kind = "Irregular"
    elif word.endswith("ing"):
        root = word[:-3] + "e"
        path = "q0 -> q1 -> q2 -> qf"
        breakdown = "write + ing (drop e)"
        kind = "Regular"
    elif word.endswith("s"):
        root = word[:-1]
        path = "q0 -> q1 -> q2 -> qf"
        breakdown = "write + s"
        kind = "Regular"
    print(f"{word:<12}{path:<25}{breakdown:<28}{root:<10}{kind}")
```

EXPECTED OUTPUT

```
Word        State Path              Breakdown                   Root      Type
------------------------------------------------------------------------------------------
writes      q0 -> q1 -> q2 -> qf    write + s                   write     Regular
writing     q0 -> q1 -> q2 -> qf    write + ing (drop e)        write     Regular
written     q0 -> q3 -> qf         Irregular past participle   write     Irregular
```

OUTPUT



Conclusion: The finite-state parser maps all three forms to write. It handles regular suffix rules and uses a lookup table for irregular verb forms, improving consistency in text processing.

Q5. Porter Stemmer-Based Processing of relational, relation, and relate

Problem statement

Implement a Porter Stemmer-based preprocessing module for document retrieval using relational, relation, and relate. Apply stemming rules sequentially, show the applied rules and intermediate forms, remove appropriate derivational suffixes, and produce the final normalized stems using Python.

1. STEP-BY-STEP STEMMING ANALYSIS

Input word	Rule applied	Intermediate form	Final stem
relational	ational → ate	relate	relat
relation	ation → ate	relate	relat
relate	Remove final e	relat	relat

2. DETAILED EXPLANATION

Word 1: relational

- Original word: relational
- Porter Step 2 replaces ational with ate.
- Intermediate form: relate.
- Porter Step 5 removes the final e.
- Final stem: relat.

Word 2: relation

- Original word: relation
- Porter Step 2 replaces ation with ate.
- Intermediate form: relate.
- Porter Step 5 removes the final e.
- Final stem: relat.

Word 3: relate

- Original word: relate
- Porter Step 5 removes the final e.
- Final stem: relat.

All three words produce the same stem, relat.

CODE

```
#JOTHIKA 192324235
from nltk.stem import PorterStemmer
ps = PorterStemmer()
words = ["relational", "relation", "relate"]
print(f"{'Word':<15}{'Rule Applied':<22}{'Intermediate':<18}{'Final Stem'}")
print("-" * 70)
for word in words:
    stem = ps.stem(word)
    if word == "relational":
        rule = "ational -> ate; remove e"
        intermediate = "relate -> relat"
    elif word == "relation":
        rule = "ation -> ate; remove e"
        intermediate = "relate -> relat"
    else:
        rule = "Remove final e"
        intermediate = "relat"
    print(f"{word:<15}{rule:<22}{intermediate:<18}{stem}")
```

EXPECTED OUTPUT

Word Rule Applied Intermediate Final Stem

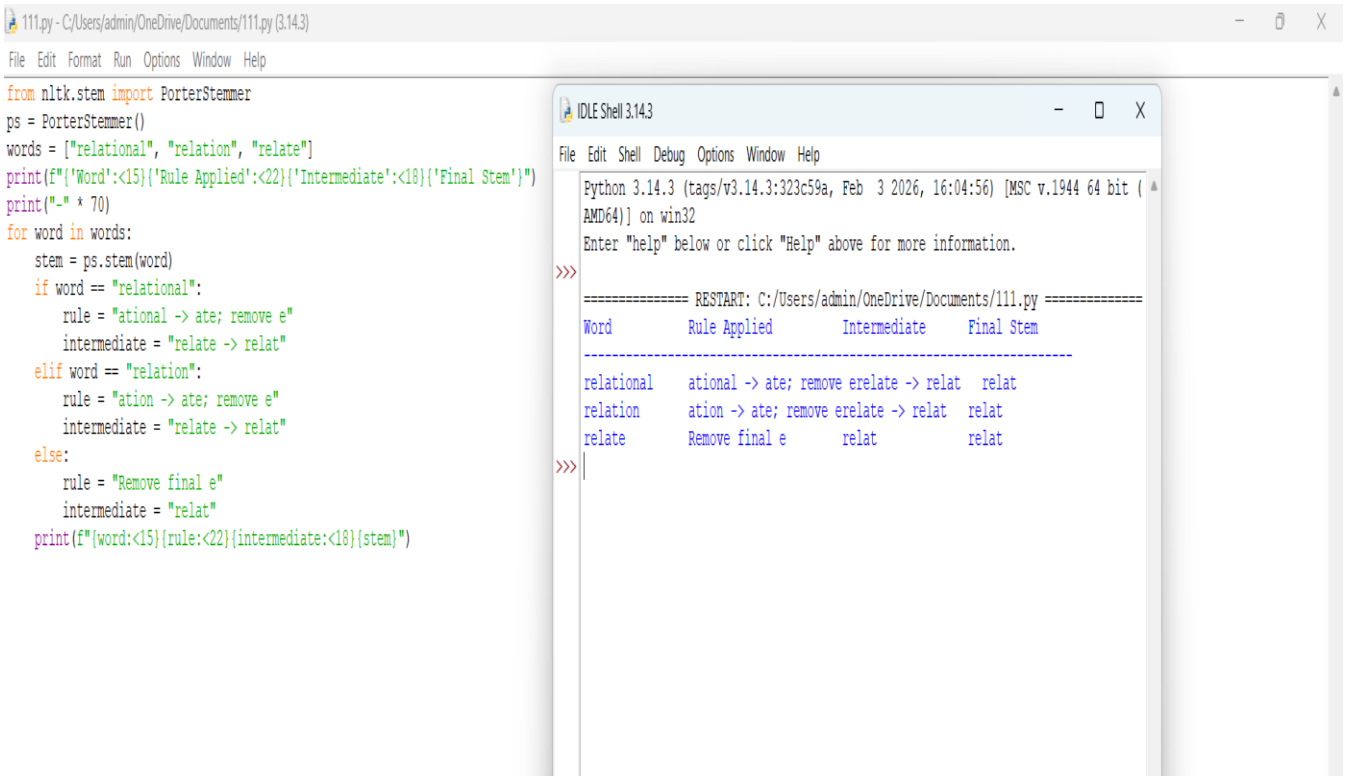
---

relational ational -> ate; remove e relate -> relat relat

relation ation -> ate; remove e relate -> relat relat

relate Remove final e relat relat

OUTPUT



Conclusion: The Porter Stemmer normalizes all three words to relat. This helps a document retrieval system match related word forms without requiring an exact word match.